IAM-12: API Provisioning & Validation Scripts

> **Series:** IAM — IAM Administration | **Notebook:** 12 of 12 | **Created:** March 2026 | **Last Updated:** 11/10/2026

Overview

This notebook collects the shell scripts and DQL queries needed to **provision**, **validate**, and **troubleshoot** a persona-based IAM model via the Dynatrace Account Management API.

It is the companion to **IAM-11: Policy Persona Workshop**, which designs the permissions model. This notebook automates the execution of that design.

> **Script-first approach:** Every script is self-contained with a CONFIGURATION block at the top. Copy, edit the variables, and run — no other files needed.

Table of Contents

1. Prerequisites & OAuth Setup
2. Script 1: End-to-End Provisioning
3. Script 2: Policy & Binding Report
4. Script 3: Cleanup Test Resources
5. DQL Validation Queries
6. API Reference
7. Troubleshooting

Prerequisites

Requirement	Details
Dynatrace Account	Account Management access (account admin or delegated)

Requirement	Details
OAuth Client	Created in Account Management → IAM → OAuth clients
OAuth Scopes	`account-idm-read`, `account-idm-write`, `iam-policies-management`
Tools	`curl`, `jq` (install with `brew install jq` on macOS)
Prior Reading	**IAM-10** (Templated Policies), **IAM-11** (Persona Workshop)

1. Prerequisites & OAuth Setup

The IAM Account Management API requires an **OAuth2 bearer token** — raw API tokens (`dt0c01.*`) will not work. You must exchange OAuth client credentials for a short-lived bearer token.

Creating an OAuth Client

1. Go to **Account Management** → **Identity & Access Management** → **OAuth clients**
2. Click **Create client**
3. Grant these scopes:
   - `account-idm-read` — Read groups and users
   - `account-idm-write` — Create/delete groups
   - `iam-policies-management` — Create/delete policies and bindings
4. Save the **Client ID** and **Client Secret**

Token Exchange Script

```
#!/usr/bin/env bash
# Exchange OAuth client credentials for a bearer token
# Usage: source this script, then use ${TOKEN} in subsequent API calls

SSO_TOKEN_URL="https://sso.dynatrace.com/sso/oauth2/token"
OAUTH_CLIENT_ID="dt0s02.XXXXXXXX"          # Your client ID
OAUTH_CLIENT_SECRET="dt0s02.XXXXXXXX.YYY"  # Your client secret
ACCOUNT_UUID="your-account-uuid"
OAUTH_SCOPE="account-idm-read account-idm-write iam-policies-management"

TOKEN_RESPONSE=$(curl -s \
  --request POST "${SSO_TOKEN_URL}" \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "grant_type=client_credentials" \
  --data-urlencode "client_id=${OAUTH_CLIENT_ID}" \
  --data-urlencode "client_secret=${OAUTH_CLIENT_SECRET}" \
  --data-urlencode "scope=${OAUTH_SCOPE}" \
  --data-urlencode "resource=urn:dtaccount:${ACCOUNT_UUID}")

TOKEN=$(echo "${TOKEN_RESPONSE}" | jq -r '.access_token // empty')
EXPIRES_IN=$(echo "${TOKEN_RESPONSE}" | jq -r '.expires_in // "unknown"')

if [[ -z "${TOKEN}" ]]; then
  echo "ERROR: Failed to acquire token"
  echo "${TOKEN_RESPONSE}" | jq .
else
  echo "Token acquired (expires in ${EXPIRES_IN}s, ${#TOKEN} chars)"
fi
```

> **Token lifetime:** OAuth tokens typically expire in 300 seconds (5 minutes). For long-running scripts, re-acquire before expiration.

2. Script 1: End-to-End Provisioning

The following shell script demonstrates the complete provisioning workflow — from creating groups and policies through binding everything together. It implements the four-persona model designed in **IAM-11**.

> **Important:** Replace all dummy values in the **CONFIGURATION** section with your actual values before running. Every customizable value is defined in one block at the top.

What the Script Creates

Step	Resource	Count	Details
1	Groups	4	Standard, PowerUser, SRE, Admin
2	Static policies	9	4 UI + 4 Config + 1 Admin-Data
3	Templated data policy	1	`tpl-team-data-reader` with `${bindParam:team}`
4	Bindings	14	4 UI + 4 Config + 1 Admin-Data + 5 Templated-Data
5	Verification	—	List all policies and bindings

API Endpoints Used

Operation	Method	Endpoint
Create group	POST	`/iam/v1/accounts/{accountUuid}/groups`
Create policy	POST	`/iam/v1/repo/{levelType}/{levelId}/policies`
Bind policy to group	POST	`/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}`
List policies	GET	`/iam/v1/repo/{levelType}/{levelId}/policies`
List bindings	GET	`/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}`

The Script

```
#!/usr/bin/env bash
# ================================================================================
# IAM End-to-End Provisioning Script
# ================================================================================
# Creates groups, policies (UI + Data + Config), and bindings for a
# four-persona model: Standard User, Power User, SRE, and Admin.
#
# Prerequisites:
#   - curl and jq installed
#   - OAuth bearer token (exchange client credentials at
#     https://sso.dynatrace.com/sso/oauth2/token)
#   - Required scopes: account-idm-read, account-idm-write, iam-policies-management
#
# Usage:
#   1. Edit the CONFIGURATION section below (all values are there)
#   2. chmod +x provision-iam.sh
#   3. ./provision-iam.sh
#
# Reference: IAM-10 (Templated Policy Assignments), IAM-04 (Policy Authoring)
# ================================================================================
# No `set -e`: each API error is handled below so that a re-run converges —
# existing groups and policies are reused, "already exists" bindings count as done,
# and any real failure is counted and reported at the end (exit 1).
set -uo pipefail

# ################################################################################
#                         CONFIGURATION
# ################################################################################
# Edit ONLY this section. The execution logic below does not need changes.
# ################################################################################

# --- Connection ---------------------------------------------------------
ACCOUNT_UUID="1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d"      # Dynatrace account UUID
ENVIRONMENT_ID="abc12345"                                  # Environment ID
API_BASE="https://api.dynatrace.com"                       # Account Management API base
TOKEN="REPLACE_WITH_OAUTH_BEARER_TOKEN"                     # See note below on
obtaining this

# --- Personas -----------------------------------------------------------
# Parallel arrays: same index = same persona. Add/remove personas by editing
# all four arrays consistently.
PERSONA_LABELS=("Standard"      "PowerUser"      "SRE"           "Admin")
PERSONA_SCOPES=("GLOBAL"        "GLOBAL"         "GLOBAL"        "GLOBAL")

GROUP_NAMES=(
  "DT-GLOBAL-StandardUsers"
  "DT-GLOBAL-PowerUsers"
  "DT-GLOBAL-SREs"
  "DT-GLOBAL-Admins"
)
GROUP_DESCRIPTIONS=(
  "Standard Users — view dashboards and run queries"
```

```
  "Power Users — modify alerting, create notebooks and workflows"
  "SREs — full ops access scoped by team"
  "Admins — full platform management (break-glass)"
)

# --- UI Policies (one statementQuery per persona) ----------------------------
# Controls which apps each persona sees and what they can do with documents.
# Uses implicit deny — apps/actions not listed are hidden/blocked.
# environment:roles:viewer is granted here UNBOUNDED, so classic entity views are
# environment-wide for every persona. This script scopes Grail data only. If a persona
# must also be management-zone scoped (IAM-11 Data-Classic), remove the viewer role
# from its entry and grant it in a separate policy bound with an MZ boundary.
STANDARD_APPS='"dynatrace.dashboards", "dynatrace.notebooks"'
POWER_APPS='"dynatrace.dashboards", "dynatrace.notebooks", "dynatrace.automations"'

UI_POLICIES=(
  # Standard: view dashboards and notebooks only
  "ALLOW environment:roles:viewer; ALLOW app-engine:apps:run WHERE shared:app-id IN
(${STANDARD_APPS}); ALLOW document:documents:read;"
  # Power User: create dashboards, notebooks, and workflows
  "ALLOW environment:roles:viewer; ALLOW app-engine:apps:run WHERE shared:app-id IN
(${POWER_APPS}); ALLOW document:documents:read; ALLOW document:documents:write;"
  # SRE: all apps, full document control
  "ALLOW environment:roles:viewer; ALLOW app-engine:apps:run; ALLOW
document:documents:read; ALLOW document:documents:write; ALLOW document:documents:delete;"
  # Admin: all apps, full document control + sharing
  "ALLOW environment:roles:viewer; ALLOW app-engine:apps:run; ALLOW
document:documents:read; ALLOW document:documents:write; ALLOW document:documents:delete;
ALLOW document:direct-shares:write;"
)

# --- Config Policies (one statementQuery per persona) ------------------------
# Controls which settings schemas each persona can read/write.
# Add schema ID prefixes (builtin:…) or schema groups (group:…) to grant write access;
read is granted to all.
POWER_SCHEMAS=(
  "builtin:davis.anomaly-detectors"   # Latest Dynatrace: custom alerts
  "builtin:alerting.profile"          # Dynatrace Classic only (not visible in Latest)
  "builtin:problem.notifications"     # Dynatrace Classic only (Latest: workflows)
  "group:preferences"
)
SRE_SCHEMAS=(
  "group:host-monitoring"
  "group:anomaly-detection"
  "group:failure-detection"
  "group:processes-and-containers"
)

# Build config policy statements from schema arrays
_build_config_policy() {
  local base="ALLOW settings:objects:read; ALLOW settings:schemas:read;"
  local schemas=("$@")
  for prefix in "${schemas[@]}"; do
    if [[ "${prefix}" == group:* ]]; then
      # Schema groups are a separate condition (operators = and IN only)
```

```
      base+=" ALLOW settings:objects:write WHERE settings:schemaGroup = \"${prefix}\";"
    else
      base+=" ALLOW settings:objects:write WHERE settings:schemaId startsWith
\"${prefix}\";"
    fi
  done
  echo "${base}"
}

CONFIG_POLICIES=(
  # Standard: read-only (write implicitly denied)
  "ALLOW settings:objects:read; ALLOW settings:schemas:read;"
  # Power User: read all + write specific schemas
  "$(_build_config_policy "${POWER_SCHEMAS[@]}")"
  # SRE: read all + write monitoring schemas
  "$(_build_config_policy "${SRE_SCHEMAS[@]}")"
  # Admin: full settings access
  "ALLOW settings:objects:read; ALLOW settings:schemas:read; ALLOW
settings:objects:write;"
)

# --- Data Policies -----------------------------------------------------------
# Templated policy: one template, bound per-team with ${bindParam:team}
TPL_DATA_NAME="tpl-team-data-reader"
TPL_DATA_DESC="Parameterized: team-scoped read access to logs, spans, metrics, and events"
TPL_DATA_STATEMENT='ALLOW storage:buckets:read; ALLOW storage:logs:read WHERE
storage:dt.security_context = "${bindParam:team}"; ALLOW storage:spans:read WHERE
storage:dt.security_context = "${bindParam:team}"; ALLOW storage:metrics:read; ALLOW
storage:events:read;'

# Admin data policy: unrestricted (no security context filter)
ADMIN_DATA_NAME="GLOBAL-Admin-Data"
ADMIN_DATA_DESC="Admins: unrestricted data access across all buckets and security
contexts"
ADMIN_DATA_STATEMENT="ALLOW storage:buckets:read; ALLOW storage:logs:read; ALLOW
storage:spans:read; ALLOW storage:metrics:read; ALLOW storage:events:read; ALLOW
storage:fieldsets:read;"

# --- Team Bindings -----------------------------------------------------------
# Teams for SRE templated data policy (SREs get all teams)
TEAMS=("checkout" "payments" "catalog")

# Default team for Standard and Power User data bindings
STANDARD_DEFAULT_TEAM="checkout"
POWER_DEFAULT_TEAM="checkout"

# ##############################################################################
#                    EXECUTION — no edits needed below
# ##############################################################################

# ------------------------------------------------------------------------------
# HELPER FUNCTIONS
# ------------------------------------------------------------------------------
api_post() {
  local endpoint="$1"
```

```
  local payload="$2"
  local description="$3"

  echo "  Creating: ${description}..." &gt;&amp;2
  RESPONSE=$(curl -s -w "\n%{http_code}" -X POST \
    "${API_BASE}${endpoint}" \
    -H "Authorization: Bearer ${TOKEN}" \
    -H "Content-Type: application/json" \
    -d "${payload}")

  HTTP_CODE=$(echo "${RESPONSE}" | tail -1)
  BODY=$(echo "${RESPONSE}" | sed '$d')

  if [[ "${HTTP_CODE}" =~ ^2 ]]; then
    echo "    OK (HTTP ${HTTP_CODE})" &gt;&amp;2
    echo "${BODY}"
  elif [[ "${HTTP_CODE}" == "400" &amp;&amp; "${BODY}" == *"already exists"* ]]; then
    # Re-run: the binding is already in the desired state — converged, not failed
    echo "    Already exists — converged (HTTP 400)" &gt;&amp;2
  else
    echo "    FAILED (HTTP ${HTTP_CODE}): ${BODY}" &gt;&amp;2
    return 1
  fi
}

api_get() {
  local endpoint="$1"
  curl -s -X GET "${API_BASE}${endpoint}" \
    -H "Authorization: Bearer ${TOKEN}" \
    -H "Content-Type: application/json"
}

extract_uuid() {
  echo "$1" | jq -r '.uuid // .id // empty' 2&gt;/dev/null
}

FAILURES=0
note_failure() { FAILURES=$((FAILURES + 1)); }

# Look up an existing object by exact name, so a re-run reuses it instead of failing
# on "Duplicate entry" or creating a second policy with the same name.
# GET groups returns {"count": N, "items": [...]}; GET policies returns {"policies":
[...]}.
group_uuid_by_name() {
  echo "${EXISTING_GROUPS}" | jq -r --arg n "$1" \
    '(.items // [])[] | select(.name == $n) | .uuid' 2&gt;/dev/null | head -n 1
}
policy_uuid_by_name() {
  echo "${EXISTING_POLICIES}" | jq -r --arg n "$1" \
    '(.policies // [])[] | select(.name == $n) | .uuid' 2&gt;/dev/null | head -n 1
}

# Create a policy unless one with this name exists; print its UUID either way.
# An existing policy is reused as-is: its statement is NOT updated.
ensure_policy() {
```

```
  local name="$1" desc="$2" stmt="$3" uuid payload result
  uuid=$(policy_uuid_by_name "${name}")
  if [[ -n "${uuid}" ]]; then
    echo "  Exists: ${name} — reusing ${uuid} (statement not updated)" &gt;&amp;2
    echo "${uuid}"
    return 0
  fi
  payload=$(jq -n --arg name "${name}" --arg desc "${desc}" --arg stmt "${stmt}" \
    '{name: $name, description: $desc, statementQuery: $stmt}')
  result=$(api_post "/iam/v1/repo/environment/${ENVIRONMENT_ID}/policies" "${payload}"
"${name}") || return 1
  extract_uuid "${result}"
}

PERSONA_COUNT=${#PERSONA_LABELS[@]}

# ------------------------------------------------------------------------
# PHASE 1: CREATE GROUPS
# ------------------------------------------------------------------------
echo ""
echo "=============================="
echo "PHASE 1: Creating Groups"
echo "=============================="

declare -a GROUP_UUIDS

EXISTING_GROUPS=$(api_get "/iam/v1/accounts/${ACCOUNT_UUID}/groups")

# The groups API expects an array payload, not a single object.
# Only groups that do not exist yet are sent, so a re-run never hits "Duplicate entry".
GROUPS_PAYLOAD=$(jq -n '[]')
for i in $(seq 0 $((PERSONA_COUNT - 1))); do
  if [[ -n "$(group_uuid_by_name "${GROUP_NAMES[$i]}")" ]]; then
    echo "  Exists: ${GROUP_NAMES[$i]} — reusing" &gt;&amp;2
    continue
  fi
  GROUPS_PAYLOAD=$(echo "${GROUPS_PAYLOAD}" | jq \
    --arg name "${GROUP_NAMES[$i]}" \
    --arg desc "${GROUP_DESCRIPTIONS[$i]}" \
    '. + [{name: $name, description: $desc}]')
done

NEW_GROUP_COUNT=$(echo "${GROUPS_PAYLOAD}" | jq 'length')
if [[ "${NEW_GROUP_COUNT}" -gt 0 ]]; then
  api_post "/iam/v1/accounts/${ACCOUNT_UUID}/groups" "${GROUPS_PAYLOAD}" \
    "${NEW_GROUP_COUNT} new group(s)" &gt;/dev/null || note_failure
  EXISTING_GROUPS=$(api_get "/iam/v1/accounts/${ACCOUNT_UUID}/groups")
fi

# Resolve every persona's group UUID by name — new and pre-existing alike
for i in $(seq 0 $((PERSONA_COUNT - 1))); do
  GROUP_UUIDS[$i]=$(group_uuid_by_name "${GROUP_NAMES[$i]}")
  if [[ -z "${GROUP_UUIDS[$i]}" ]]; then
    echo "ERROR: group ${GROUP_NAMES[$i]} not found — cannot bind policies to it.
Stopping." &gt;&amp;2
```

```
    exit 1
  fi
done

echo ""
echo "Groups (created or reused):"
for i in $(seq 0 $((PERSONA_COUNT - 1))); do
  printf "  %-12s %s\n" "${PERSONA_LABELS[$i]}:" "${GROUP_UUIDS[$i]}"
done

# ------------------------------------------------------------------------
# PHASE 2: CREATE POLICIES
# ------------------------------------------------------------------------
echo ""
echo "============================="
echo "PHASE 2: Creating Policies"
echo "============================="

declare -a UI_UUIDS CONFIG_UUIDS

EXISTING_POLICIES=$(api_get "/iam/v1/repo/environment/${ENVIRONMENT_ID}/policies")

for i in $(seq 0 $((PERSONA_COUNT - 1))); do
  SCOPE="${PERSONA_SCOPES[$i]}"
  LABEL="${PERSONA_LABELS[$i]}"
  UI_UUIDS[$i]=$(ensure_policy "${SCOPE}-${LABEL}-UI" "${LABEL} UI policy"
"${UI_POLICIES[$i]}") || note_failure
  CONFIG_UUIDS[$i]=$(ensure_policy "${SCOPE}-${LABEL}-Config" "${LABEL} Config policy"
"${CONFIG_POLICIES[$i]}") || note_failure
done

# Templated data policy
TPL_DATA_UUID=$(ensure_policy "${TPL_DATA_NAME}" "${TPL_DATA_DESC}"
"${TPL_DATA_STATEMENT}") || note_failure

# Admin data policy (unrestricted)
ADMIN_DATA_UUID=$(ensure_policy "${ADMIN_DATA_NAME}" "${ADMIN_DATA_DESC}"
"${ADMIN_DATA_STATEMENT}") || note_failure

echo ""
echo "Policies (created or reused):"
for i in $(seq 0 $((PERSONA_COUNT - 1))); do
  printf "  %-12s UI=%s  Config=%s\n" "${PERSONA_LABELS[$i]}:" "${UI_UUIDS[$i]}"
"${CONFIG_UUIDS[$i]}"
done
echo "  Data:        Template=${TPL_DATA_UUID}  Admin=${ADMIN_DATA_UUID}"

# ------------------------------------------------------------------------
# PHASE 3: BIND STATIC POLICIES TO GROUPS
# ------------------------------------------------------------------------
echo ""
echo "============================="
echo "PHASE 3: Binding Static Policies"
echo "============================="
```

```
for i in $(seq 0 $((PERSONA_COUNT - 1))); do
  LABEL="${PERSONA_LABELS[$i]}"

  # Bind UI
  api_post "/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${UI_UUIDS[$i]}" \
    "{\"groups\":[\"${GROUP_UUIDS[$i]}\"]}" \
    "${LABEL}-UI → ${GROUP_NAMES[$i]}" || note_failure

  # Bind Config
  api_post "/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${CONFIG_UUIDS[$i]}" \
    "{\"groups\":[\"${GROUP_UUIDS[$i]}\"]}" \
    "${LABEL}-Config → ${GROUP_NAMES[$i]}" || note_failure
done

# Bind Admin data policy (unrestricted — last persona is Admin)
ADMIN_IDX=$((PERSONA_COUNT - 1))
api_post "/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${ADMIN_DATA_UUID}" \
  "{\"groups\":[\"${GROUP_UUIDS[$ADMIN_IDX]}\"]}" \
  "Admin-Data → ${GROUP_NAMES[$ADMIN_IDX]}" || note_failure

# ------------------------------------------------------------------------
# PHASE 4: BIND TEMPLATED DATA POLICY PER TEAM
# ------------------------------------------------------------------------
echo ""
echo "============================="
echo "PHASE 4: Binding Templated Data Policy Per Team"
echo "============================="

# SREs get all teams (index 2 = SRE)
SRE_IDX=2
for team in "${TEAMS[@]}"; do
  api_post "/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${TPL_DATA_UUID}" \
    "{\"groups\":[\"${GROUP_UUIDS[$SRE_IDX]}\"],\"parameters\":{\"team\":\"${team}\"}}" \
    "${TPL_DATA_NAME} (team=${team}) → ${GROUP_NAMES[$SRE_IDX]}" || note_failure
done

# Standard Users get their default team (index 0 = Standard)
api_post "/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${TPL_DATA_UUID}" \
  "{\"groups\":[\"${GROUP_UUIDS[0]}\"],\"parameters\":
{\"team\":\"${STANDARD_DEFAULT_TEAM}\"}}" \
  "${TPL_DATA_NAME} (team=${STANDARD_DEFAULT_TEAM}) → ${GROUP_NAMES[0]}" || note_failure

# Power Users get their default team (index 1 = PowerUser)
api_post "/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${TPL_DATA_UUID}" \
  "{\"groups\":[\"${GROUP_UUIDS[1]}\"],\"parameters\":
{\"team\":\"${POWER_DEFAULT_TEAM}\"}}" \
  "${TPL_DATA_NAME} (team=${POWER_DEFAULT_TEAM}) → ${GROUP_NAMES[1]}" || note_failure

# ------------------------------------------------------------------------
# PHASE 5: VERIFY
# ------------------------------------------------------------------------
echo ""
echo "============================="
echo "PHASE 5: Verification"
echo "============================="
```

```
echo ""
echo "--- All Policies ---"
api_get "/iam/v1/repo/environment/${ENVIRONMENT_ID}/policies" | jq -r \
  '.policies[] | "  \(.uuid)  \(.name)"' 2&gt;/dev/null

echo ""
echo "--- Templated Data Policy Bindings ---"
api_get "/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${TPL_DATA_UUID}" \
  | jq .

echo ""
echo "--- All Groups ---"
api_get "/iam/v1/accounts/${ACCOUNT_UUID}/groups" | jq -r \
  '.items[] | select(.name | startswith("DT-")) | "  \(.uuid)  \(.name)"' 2&gt;/dev/null

# ------------------------------------------------------------------------
# SUMMARY
# ------------------------------------------------------------------------
echo ""
echo "=============================="
echo "PROVISIONING COMPLETE"
echo "=============================="
echo ""
echo "Resources in place (created now or reused from an earlier run):"
echo "  Groups:            ${PERSONA_COUNT}"
echo "  Static policies:   $((PERSONA_COUNT * 2 + 1))  (${PERSONA_COUNT} UI +
${PERSONA_COUNT} Config + 1 Admin-Data)"
echo "  Template policies: 1  (${TPL_DATA_NAME})"
BINDING_COUNT=$((PERSONA_COUNT * 2 + 1 + ${#TEAMS[@]} + 2))
echo "  Bindings:         ${BINDING_COUNT}"
echo ""
if [[ ${FAILURES} -gt 0 ]]; then
  echo "Finished with ${FAILURES} failed step(s) — fix the FAILED lines above and re-run."
  echo "Re-running is safe: existing groups and policies are reused, existing bindings
count as done."
  exit 1
fi
echo "Next steps:"
echo "  1. Add users to groups via SCIM sync or manual assignment"
echo "  2. Verify access by logging in as each persona"
echo "  3. Run the DQL audit query from the Validating Your Design section"
echo "  4. Export to Monaco YAML for policy-as-code (see IAM-10)"
echo ""
```

Configuration Reference

All customizable values live in the **CONFIGURATION** block at the top of the script. The execution logic below it loops through these arrays automatically — no edits needed there.

Variable	Purpose	Example
`ACCOUNT_UUID`	Your Dynatrace account UUID	`1a2b3c4d-...`

Variable	Purpose	Example
`ENVIRONMENT_ID`	Target environment ID	`abc12345`
`API_BASE`	Account Management API base URL	`https://api.dynatrace.com`
`TOKEN`	OAuth bearer token (see Prerequisites)	Exchange at SSO endpoint
`PERSONA_LABELS`	Short name for each persona (used in policy names)	`("Standard" "PowerUser" "SRE" "Admin")`
`PERSONA_SCOPES`	Naming prefix per persona	`("GLOBAL" "GLOBAL" "GLOBAL" "GLOBAL")`
`GROUP_NAMES`	Dynatrace group name per persona	`("DT-GLOBAL-StandardUsers" ...)`
`GROUP_DESCRIPTIONS`	Human-readable group description	`("Standard Users — ..." ...)`
`STANDARD_APPS` / `POWER_APPS`	App IDs visible to Standard / Power users	`"dynatrace.dashboards", "dynatrace.notebooks"`
`UI_POLICIES`	Full `statementQuery` per persona for UI access	One entry per persona
`POWER_SCHEMAS` / `SRE_SCHEMAS`	Schema prefixes each persona can write	`("builtin:alerting.profile" ...)`
`CONFIG_POLICIES`	Auto-built from schema arrays (or override directly)	One entry per persona
`TPL_DATA_STATEMENT`	Templated data policy with `${bindParam:team}`	`ALLOW storage:logs:read WHERE ...`
`ADMIN_DATA_STATEMENT`	Unrestricted admin data policy	`ALLOW storage:logs:read; ...`
`TEAMS`	Team names for SRE templated bindings	`("checkout" "payments" "catalog")`
`STANDARD_DEFAULT_TEAM`	Default team for Standard User data binding	`"checkout"`
`POWER_DEFAULT_TEAM`	Default team for Power User data binding	`"checkout"`

Script Design Notes

Why no DENY statements? Every policy uses implicit deny — permissions not ALLOWed are denied by default. This keeps the model composable: a user in both StandardUsers and PowerUsers inherits

the union of all ALLOWs from both groups without any DENY conflicts. See the Tips and Key Principles section in **IAM-11** for details.

Why one templated data policy? Rather than creating separate data policies for Standard, Power, and SRE personas, they all share `tpl-team-data-reader`. The difference is in the **binding** — each group is bound with a different `team` parameter value. Admin uses a separate unrestricted policy because it needs access to all security contexts.

Why `storage:buckets:read` in both data policies? It is *"Required additionally to a table permission"* — without it, `storage:logs:read` and the rest return no records. It takes no `dt.security_context` condition, so it is granted unconditionally and the table permissions carry the team scope.

Why the viewer role is unbounded here. Every UI policy grants `environment:roles:viewer` with no boundary, so classic entity views are environment-wide for all four personas — the script scopes Grail data only. For a persona that must also be management-zone scoped, follow **IAM-11** *Writing Policies*: take the viewer role out of that persona's `UI_POLICIES` entry and grant it only in a separate policy bound with a management-zone boundary. Leaving it in the UI policy as well would give the role environment-wide and undo the boundary.

Re-running is safe. The script looks up groups and policies by name before creating them and reuses any that exist, so a second run does not fail on *"Duplicate entry"*. A binding that already exists returns HTTP 400 *"Policy binding cannot be created, such binding already exists"*, which the script reports as converged. Real failures are counted and the script exits 1 at the end. A reused policy keeps its existing statement — edit it in place (or delete it) when you change a statement here.

> **Sources:** IAM policy statements (DT docs) — `storage:buckets:read`: *"Required additionally to a table permission."*, List all groups (DT docs) — response `GroupListDto` with `count` and `items`.

Adapting for your organization:
- **Add a persona** — append to all parallel arrays (`PERSONA_LABELS`, `GROUP_NAMES`, `GROUP_DESCRIPTIONS`, `UI_POLICIES`, `CONFIG_POLICIES`) and the execution loop handles it automatically
- **Change visible apps** — edit `STANDARD_APPS` or `POWER_APPS`, or modify `UI_POLICIES` entries directly
- **Change writable schemas** — edit `POWER_SCHEMAS` or `SRE_SCHEMAS` arrays; the `_build_config_policy` function generates the policy statement. `builtin:alerting.profile` and `builtin:problem.notifications` exist only on **Dynatrace Classic** — they are not visible in Latest Dynatrace, where problem notifications are workflows (`automation:workflows:*`, IAM-04 Pattern 6), not a settings schema. Remove them once every target environment runs Latest Dynatrace.

> **Sources:** Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs) — *"None of the schemas on this page are visible in Latest Dynatrace."*
> - **Add teams** — append to the `TEAMS` array
> - **Per-team SRE groups** — create separate groups (`DT-SRE-Checkout`, `DT-SRE-Payments`) and adjust the Phase 4 loop to bind each team to its own group
> - **Bucket isolation** — add `AND storage:bucket-name = "${bindParam:bucket}"` to `TPL_DATA_STATEMENT` and change its `ALLOW storage:buckets:read;` to `ALLOW`

> `storage:buckets:read WHERE storage:bucket-name = "${bindParam:bucket}";` (see **IAM-10** Pattern 4)

3. Script 2: Policy & Binding Report

The Dynatrace UI does not always surface templated policy bindings with their parameter values. This script queries the API and produces a formatted report showing every policy, its statement, and all bindings with resolved group names and parameters.

```
#!/usr/bin/env bash
# ================================================================================
# Policy &amp;amp; Binding Report — shows templated parameters the GUI may hide
# ================================================================================
# Prerequisites: TOKEN, ACCOUNT_UUID, ENVIRONMENT_ID set (see Section 1)
# ================================================================================
set -uo pipefail

# --- CONFIGURATION -----------------------------------------------------------
ACCOUNT_UUID="your-account-uuid"
ENVIRONMENT_ID="your-environment-id"
API_BASE="https://api.dynatrace.com"
TOKEN="your-oauth-bearer-token"              # See Section 1 for token exchange

# Optional: filter to only show policies matching a prefix (empty = show all)
POLICY_FILTER=""   # e.g., "GLOBAL-" or "tpl-" or "" for all custom policies

# --- COLORS ------------------------------------------------------------------
CYAN='\033[0;36m'
GREEN='\033[0;32m'
YELLOW='\033[0;33m'
RED='\033[0;31m'
NC='\033[0m'

# --- FETCH GROUPS (for UUID → name resolution) -------------------------------
echo ""
echo "Fetching groups..."
ALL_GROUPS=$(curl -s -X GET \
  "${API_BASE}/iam/v1/accounts/${ACCOUNT_UUID}/groups" \
  -H "Authorization: Bearer ${TOKEN}")

group_name() {
  echo "${ALL_GROUPS}" | jq -r --arg u "$1" '
    (if type == "array" then . else (.items // .groups // []) end)[]
    | select(.uuid == $u) | .name' 2&amp;gt;/dev/null
}

# --- FETCH POLICIES ----------------------------------------------------------
echo "Fetching policies..."
ALL_POLICIES=$(curl -s -X GET \
  "${API_BASE}/iam/v1/repo/environment/${ENVIRONMENT_ID}/policies" \
  -H "Authorization: Bearer ${TOKEN}")

POLICY_COUNT=$(echo "${ALL_POLICIES}" | jq '(.policies // []) | length' 2&amp;gt;/dev/null)
echo "Found ${POLICY_COUNT} environment-level policies."

# --- REPORT ------------------------------------------------------------------
echo ""
echo "================================================"
echo "  POLICY &amp;amp; BINDING REPORT"
echo "  Environment: ${ENVIRONMENT_ID}"
echo "================================================"

echo "${ALL_POLICIES}" | jq -c '(.policies // [])[]' 2&amp;gt;/dev/null | while IFS= read -r
policy; do
```

```
  PUUID=$(echo "${policy}" | jq -r '.uuid')
  PNAME=$(echo "${policy}" | jq -r '.name')
  PSTMT=$(echo "${policy}" | jq -r '.statementQuery // "(no statement)"')
  PTAGS=$(echo "${policy}" | jq -r '(.tags // []) | join(", ")')

  # Apply filter if set
  if [[ -n "${POLICY_FILTER}" ]] &amp;&amp; [[ ! "${PNAME}" == ${POLICY_FILTER}* ]]; then
    continue
  fi

  # Skip Dynatrace built-in policies (they have no statementQuery we can edit)
  if echo "${policy}" | jq -e '.builtIn == true' &amp;&gt;/dev/null; then
    continue
  fi

  IS_TEMPLATE="no"
  if echo "${PSTMT}" | grep -q 'bindParam'; then
    IS_TEMPLATE="YES"
  fi

  echo ""
  echo -e "  ${CYAN}Policy:${NC}     ${PNAME}"
  echo -e "  ${CYAN}UUID:${NC}       ${PUUID}"
  echo -e "  ${CYAN}Templated:${NC}  ${IS_TEMPLATE}"
  [[ -n "${PTAGS}" ]] &amp;&amp; echo -e "  ${CYAN}Tags:${NC}       ${PTAGS}"

  echo -e "  ${CYAN}Statement:${NC}"
  echo "${PSTMT}" | tr ';' '\n' | sed 's/^ *//' | grep -v '^$' | sed 's/^/    /'

  # Fetch bindings
  BIND_RESPONSE=$(curl -s -X GET \
    "${API_BASE}/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${PUUID}" \
    -H "Authorization: Bearer ${TOKEN}")

  BINDING_COUNT=$(echo "${BIND_RESPONSE}" | jq '(.policyBindings // []) | length'
2&gt;/dev/null)

  if [[ "${BINDING_COUNT}" == "0" ]] || [[ -z "${BINDING_COUNT}" ]]; then
    echo -e "  ${YELLOW}Bindings:${NC}   (none — policy is unbound)"
  else
    echo -e "  ${CYAN}Bindings:${NC}   ${BINDING_COUNT}"

    echo "${BIND_RESPONSE}" | jq -c '.policyBindings[]' 2&gt;/dev/null | while IFS= read -
r binding; do
      B_GROUPS=$(echo "${binding}" | jq -r '.groups[]' 2&gt;/dev/null)
      B_PARAMS=$(echo "${binding}" | jq -r '
        .parameters // {} | to_entries[]
        | "\(.key)=\(.value)"' 2&gt;/dev/null)

      while IFS= read -r gid; do
        [[ -z "${gid}" ]] &amp;&amp; continue
        GNAME=$(group_name "${gid}")
        if [[ -n "${B_PARAMS}" ]]; then
          echo -e "    → ${GREEN}${GNAME:-${gid}}${NC}  params: ${YELLOW}${B_PARAMS}${NC}"
        else
```

```
          echo -e "    → ${GREEN}${GNAME:-${gid}}${NC}"
        fi
      done &lt;&lt;&lt; "${B_GROUPS}"
    done
  fi

  echo "  ──────────────────────────────────────────────"
done

echo ""
echo "Report complete."
```

Example Output

```
  Policy:     tpl-team-data-reader
  UUID:       a1b2c3d4-...
  Templated:  YES
  Statement:
    ALLOW storage:buckets:read
    ALLOW storage:logs:read WHERE storage:dt.security_context = "${bindParam:team}"
    ALLOW storage:spans:read WHERE storage:dt.security_context = "${bindParam:team}"
    ALLOW storage:metrics:read
    ALLOW storage:events:read
  Bindings:   5
    → DT-GLOBAL-SREs          params: team=checkout
    → DT-GLOBAL-SREs          params: team=payments
    → DT-GLOBAL-SREs          params: team=catalog
    → DT-GLOBAL-StandardUsers params: team=checkout
    → DT-GLOBAL-PowerUsers    params: team=checkout
```

4. Script 3: Cleanup Test Resources

Use this script to remove groups, policies, and bindings created while testing in a **sandbox** account. It is deliberately conservative:

- **Dry run by default.** Without `--apply` it only lists what matches — nothing is deleted.
- **Explicit confirmation.** With `--apply` it deletes only after you type `DELETE <count>` for the exact number of resources listed; anything else, or no terminal input, aborts with nothing deleted.
- **Scoped by prefix.** It selects environment-level policies whose names start with one of `POLICY_PREFIXES` (bindings are removed first) and account groups whose names start with `GROUP_PREFIX`. Built-in policies are never selected, and an empty or very short prefix makes the script refuse to run.

```
#!/usr/bin/env bash
# ================================================================================
# IAM Cleanup — removes TEST groups, policies, and bindings by name prefix
# ================================================================================
# Usage:
#   ./IAM-CleanUp.sh           # DRY RUN (default): lists what would be deleted, deletes
nothing
#   ./IAM-CleanUp.sh --apply   # Deletes — only after you type the confirmation phrase
#
# Scope: environment-level policies (and their bindings) whose names start with
# one of POLICY_PREFIXES, and account groups whose names start with GROUP_PREFIX.
# Built-in Dynatrace policies are never selected.
# ================================================================================
set -euo pipefail

MODE="dry-run"
case "${1:-}" in
  "")      ;;
  --apply) MODE="apply" ;;
  *)       echo "Usage: $0 [--apply]"; exit 2 ;;
esac

# --- CONFIGURATION ------------------------------------------------------------
# Use prefixes that ONLY your test resources carry. Never use a production
# naming prefix here (for example GLOBAL-, the persona convention in IAM-11).
POLICY_PREFIXES=("TEST-" "tpl-test-")
GROUP_PREFIX="DT-TEST-"

SSO_TOKEN_URL="https://sso.dynatrace.com/sso/oauth2/token"
OAUTH_CLIENT_ID="dt0s02.xxxxxxx"                    # Your client ID
OAUTH_CLIENT_SECRET="dt0s02.xxxxxxx.xxxxxxxxxx"     # Your client secret
ACCOUNT_UUID="xxx-xxx-xxx-xxx-xxx"                  # Dynatrace account UUID
ENVIRONMENT_ID="YOUR_ENVIRONMENT_ID"                # Environment ID
API_BASE="https://api.dynatrace.com"                # Account Management API base
OAUTH_SCOPE="account-idm-read account-idm-write iam-policies-management"

# --- COLORS -------------------------------------------------------------------
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[0;33m'
CYAN='\033[0;36m'
NC='\033[0m'

# --- SAFETY GUARD: refuse empty or very short prefixes ------------------------
for p in "${POLICY_PREFIXES[@]}" "${GROUP_PREFIX}"; do
  if [[ ${#p} -lt 4 ]]; then
    echo -e "${RED}ERROR:${NC} prefix '${p}' is empty or shorter than 4 characters —
refusing to run."
    exit 1
  fi
done

# --- TOKEN --------------------------------------------------------------------
TOKEN_RESPONSE=$(curl -s \
  --request POST "${SSO_TOKEN_URL}" \
```

```
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "grant_type=client_credentials" \
  --data-urlencode "client_id=${OAUTH_CLIENT_ID}" \
  --data-urlencode "client_secret=${OAUTH_CLIENT_SECRET}" \
  --data-urlencode "scope=${OAUTH_SCOPE}" \
  --data-urlencode "resource=urn:dtaccount:${ACCOUNT_UUID}")

TOKEN=$(echo "${TOKEN_RESPONSE}" | jq -r '.access_token // empty')
if [[ -z "${TOKEN}" ]]; then
  echo -e "${RED}ERROR:${NC} failed to acquire token"
  echo "${TOKEN_RESPONSE}" | jq . || true
  exit 1
fi

echo ""
echo "============================================="
echo "  IAM Cleanup — mode: ${MODE}"
echo "  Policy prefixes: ${POLICY_PREFIXES[*]}"
echo "  Group prefix:    ${GROUP_PREFIX}"
echo "============================================="

# --- SELECT POLICIES ---------------------------------------------------------
POLICIES=$(curl -s -X GET \
  "${API_BASE}/iam/v1/repo/environment/${ENVIRONMENT_ID}/policies" \
  -H "Authorization: Bearer ${TOKEN}")
if ! echo "${POLICIES}" | jq -e '.policies | type == "array"' &gt;/dev/null 2&gt;&amp;1;
then
  echo -e "${RED}ERROR:${NC} could not list policies — nothing deleted."
  exit 1
fi

PREFIX_JSON=$(printf '%s\n' "${POLICY_PREFIXES[@]}" | jq -R . | jq -s .)
POLICY_ROWS=()
while IFS= read -r row; do
  [[ -n "${row}" ]] &amp;&amp; POLICY_ROWS+=("${row}")
done &lt; &lt;(echo "${POLICIES}" | jq -r --argjson pfx "${PREFIX_JSON}" '
  .policies[]
  | select(.builtIn != true)
  | select(.name as $n | any($pfx[]; . as $p | $n | startswith($p)))
  | .uuid + "|" + .name')

# --- SELECT GROUPS -----------------------------------------------------------
GROUPS_JSON=$(curl -s -X GET \
  "${API_BASE}/iam/v1/accounts/${ACCOUNT_UUID}/groups" \
  -H "Authorization: Bearer ${TOKEN}")
if ! echo "${GROUPS_JSON}" | jq -e '(if type == "array" then . else (.items // .groups)
end) | type == "array"' &gt;/dev/null 2&gt;&amp;1; then
  echo -e "${RED}ERROR:${NC} could not list groups — nothing deleted."
  exit 1
fi

GROUP_ROWS=()
while IFS= read -r row; do
  [[ -n "${row}" ]] &amp;&amp; GROUP_ROWS+=("${row}")
done &lt; &lt;(echo "${GROUPS_JSON}" | jq -r --arg p "${GROUP_PREFIX}" '
```

```
  (if type == "array" then . else (.items // .groups) end)[]
  | select(.name | startswith($p))
  | .uuid + "|" + .name')

# --- PLAN ----------------------------------------------------------------
TOTAL=$(( ${#POLICY_ROWS[@]} + ${#GROUP_ROWS[@]} ))
echo ""
echo -e "${CYAN}Policies (bindings removed first):${NC} ${#POLICY_ROWS[@]}"
for row in "${POLICY_ROWS[@]+"${POLICY_ROWS[@]}"}"; do echo "  - ${row#*|}  (${row%%|*})";
done
echo -e "${CYAN}Groups:${NC} ${#GROUP_ROWS[@]}"
for row in "${GROUP_ROWS[@]+"${GROUP_ROWS[@]}"}"; do echo "  - ${row#*|}  (${row%%|*})";
done

if [[ ${TOTAL} -eq 0 ]]; then
  echo ""
  echo "Nothing matches the configured prefixes. Nothing to do."
  exit 0
fi

if [[ "${MODE}" != "apply" ]]; then
  echo ""
  echo -e "${YELLOW}Dry run — nothing was deleted.${NC} Review the list, then re-run with
--apply."
  exit 0
fi

# --- CONFIRM -------------------------------------------------------------
echo ""
read -r -p "Type 'DELETE ${TOTAL}' to delete these ${TOTAL} resources: " CONFIRM ||
CONFIRM=""
if [[ "${CONFIRM}" != "DELETE ${TOTAL}" ]]; then
  echo "Confirmation did not match — nothing deleted."
  exit 1
fi

# --- DELETE --------------------------------------------------------------
FAILED=0
for row in "${POLICY_ROWS[@]+"${POLICY_ROWS[@]}"}"; do
  puuid="${row%%|*}"; pname="${row#*|}"
  echo -n "  Bindings of ${pname}... "
  HTTP=$(curl -s -o /dev/null -w "%{http_code}" -X DELETE \
    "${API_BASE}/iam/v1/repo/environment/${ENVIRONMENT_ID}/bindings/${puuid}?
forceMultiple=true" \
    -H "Authorization: Bearer ${TOKEN}") || HTTP="000"
  [[ "${HTTP}" =~ ^2 ]] &amp;amp;&amp;amp; echo -e "${GREEN}OK${NC}" || echo -e "${YELLOW}${HTTP}
(may have no bindings)${NC}"

  echo -n "  Policy ${pname}... "
  HTTP=$(curl -s -o /dev/null -w "%{http_code}" -X DELETE \
    "${API_BASE}/iam/v1/repo/environment/${ENVIRONMENT_ID}/policies/${puuid}?force=true" \
    -H "Authorization: Bearer ${TOKEN}") || HTTP="000"
  if [[ "${HTTP}" =~ ^2 ]]; then echo -e "${GREEN}OK${NC}"; else echo -e "${RED}FAILED
(HTTP ${HTTP})${NC}"; FAILED=$((FAILED + 1)); fi
done
```

```
for row in "${GROUP_ROWS[@]+"${GROUP_ROWS[@]}"}"; do
  guuid="${row%%|*}"; gname="${row#*|}"
  echo -n "  Group ${gname}... "
  HTTP=$(curl -s -o /dev/null -w "%{http_code}" -X DELETE \
    "${API_BASE}/iam/v1/accounts/${ACCOUNT_UUID}/groups/${guuid}" \
    -H "Authorization: Bearer ${TOKEN}") || HTTP="000"
  if [[ "${HTTP}" =~ ^2 ]]; then echo -e "${GREEN}OK${NC}"; else echo -e "${RED}FAILED
(HTTP ${HTTP})${NC}"; FAILED=$((FAILED + 1)); fi
done

echo ""
if [[ ${FAILED} -gt 0 ]]; then
  echo -e "${RED}Cleanup finished with ${FAILED} failure(s).${NC}"
  exit 1
fi
echo -e "${GREEN}Cleanup complete.${NC}"
```

> **Safety:** Set `POLICY_PREFIXES` and `GROUP_PREFIX` to prefixes that **only your test resources carry**, and always read the dry-run list before running with `--apply`. Do not point it at `GLOBAL-` or `DT-GLOBAL-` — that is the production persona naming convention (IAM-11), and it is also what Script 1 creates by default. To make Script 1's output removable by this script, set `PERSONA_SCOPES` to `TEST`, name the groups `DT-TEST-…`, and rename `TPL_DATA_NAME` / `ADMIN_DATA_NAME` to `tpl-test-…` / `TEST-…` before provisioning a sandbox.

5. DQL Validation Queries

These DQL queries let you audit your IAM configuration directly from a Dynatrace Notebook — no shell scripts needed.

> **Group, policy, boundary and binding changes are account-level, and this environment's audit events are not their change log.** *"Dynatrace provides audit logs of changes to your account-level identity and access (IAM) management settings" — "Administrators can view these logs in Account Management > Settings > Audit log"*, where they are kept for up to ten years (also queryable through the Account Audits API). The same page says *"The Account Management audit log doesn't yet cover every account-level change. Some IAM configuration and login events aren't recorded." "To view the **Audit logs** page, you need the `account-company-info` role"*; API access needs an OAuth client with the `account-audit-logs-read` scope as well. The query below shows IAM API calls made **through this environment** — useful for spotting automation, not a change log. These events show IAM API calls made through this environment, including environment-level policy-binding changes; scheduled `/lookups/iam_*` file uploads are excluded by the query. Account-level group and policy changes are recorded in the account audit log.
>
> **Sources:** Account Management audit logs (DT docs).

```
// IAM API activity in this environment (last 7 days)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in `logs`: this cell
used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source on a Grail
tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform audit
records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records over 7 days
here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-scraping is
replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"), authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE", "UPDATE"}) and
contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 25
```

Audit: Who Changed Policies Recently?

Policy, group and binding changes are account-level: answer this question from the account audit log (Account Management > Settings > Audit log — requires the `account-company-info` role), which records each change with its previous and current state. The query above complements it by showing which users and automation called the IAM API **through this environment** — look for:
- **POST / PUT / DELETE** calls against `/platform/iam/…` paths from users or clients you do not expect
- Calls with a non-2xx `event.outcome`, which point at failing provisioning scripts

Verify Group Membership

```
// IAM API activity by operation type
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in `logs`: this cell
used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source on a Grail
tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform audit
records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records over 7 days
here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-scraping is
replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"), authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "AUDIT_EVENT"
| filter contains(resource, "iam")
// Corrected 10/12/2026: the daily /lookups/iam_* upload job matched "iam" (8 of 10 events
in 24 h)
| filter not startsWith(resource, "/lookups/")
| summarize calls = count(), by:{event.type, event.outcome}
| sort calls desc
```

Verify Data Access by Security Context

Use this query to confirm that security context values match what your templated data policies expect. If a team's logs have no `dt.security_context` tag, the templated policy won't match anything.

```
// Show distinct security context values in logs (last 1h)
// These values must match the "team" parameter in your templated data policy bindings
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize log_count = count(), by:{dt.security_context}
| sort log_count desc
```

Verify Bucket-Level Access

If your data policies include bucket restrictions, verify which buckets exist, which table each belongs to, and their retention. `dt.system.buckets` lists every bucket across all tables and reads zero billable bytes. It also carries a `has_access` column; in community practice, running the query as a member of each persona group and comparing that column is a quick check of bucket grants — confirm with a record query before relying on it.

```
// List all Grail buckets with their table, retention, and record counts (corrected
10/13/2026:
// this cell used `fetch logs ... by:{dt.system.bucket}`, which lists only log buckets
that
// received data in the last hour and shows no retention)
fetch dt.system.buckets
| fields name, dt.system.table, retention_days, records, has_access
| sort dt.system.table asc, name asc
```

6. API Reference

Endpoint Summary

Operation	Method	Endpoint	Notes
Groups			
List groups	GET	`/iam/v1/accounts/{accountUuid}/groups`	Returns an object `{"count": N, "items": [...]}` — read groups from `.items[]`
Create groups	POST	`/iam/v1/accounts/{accountUuid}/groups`	**Payload must be an array** `[{...}]`
Delete group	DELETE	`/iam/v1/accounts/{accountUuid}/groups/{groupUuid}`	Returns 200
Policies			
List policies	GET	`/iam/v1/repo/{level}/{levelId}/policies`	Level: `environment`, `account`, `global`
Create policy	POST	`/iam/v1/repo/{level}/{levelId}/policies`	Returns object with `.uuid`
Delete policy	DELETE	`/iam/v1/repo/{level}/{levelId}/policies/{policyUuid}?force=true`	`force=true` if bound
Bindings			

Operation	Method	Endpoint	Notes
List bindings	GET	`/iam/v1/repo/{level}/{levelId}/bindings/{policyUuid}`	Shows groups + parameters
Create binding	POST	`/iam/v1/repo/{level}/{levelId}/bindings/{policyUuid}`	Body: `{groups, parameters}`
Delete bindings	DELETE	`/iam/v1/repo/{level}/{levelId}/bindings/{policyUuid}?forceMultiple=true`	Removes all bindings
OAuth			
Token exchange	POST	`https://sso.dynatrace.com/sso/oauth2/token`	Client credentials flow

Key API Quirks

Quirk	Details
Groups payload is an array	`POST .../groups` expects `[{name, description}]`, not `{name, description}`. Sending a single object returns 500: "payload.map is not a function"
Groups use a different path	Groups: `/iam/v1/accounts/...` — Policies/bindings: `/iam/v1/repo/...`
Use jq for policy payloads	Policy `statementQuery` contains nested quotes. Use `jq -n --arg stmt "$VAR"` instead of manual escaping
OAuth required	Raw API tokens (`dt0c01.*`) don't work. Exchange client credentials at the SSO endpoint
Token lifetime	OAuth tokens expire in ~300 seconds. Re-acquire for long scripts
Binding parameters	Templated policy parameters go in the binding body: `{"groups": [...], "parameters": {"team": "value"}}`

Valid Document Permissions

Permission	Purpose
`document:documents:read`	Read dashboards and notebooks
`document:documents:write`	Create and update (covers both)
`document:documents:delete`	Delete documents
`document:documents:admin`	Full admin (bypasses ownership)
`document:direct-shares:read`	View sharing settings

Permission	Purpose
`document:direct-shares:write`	Share documents with others
`document:direct-shares:delete`	Remove shares

> **Common mistake:** `document:documents:create` and `document:documents:share` do **not exist**. Use `write` for creation and `direct-shares:write` for sharing.

7. Troubleshooting

Common API Errors

HTTP Code	Error Message	Cause	Fix
400	"payload.map is not a function"	Groups POST received an object instead of array	Wrap payload in `[{...}]`
400	"Invalid permission name: documents:create"	Non-existent permission	Use `documents:write` (covers create)
400	"ALLOW or DENY must be followed by a permission in the form service-name:resource:action"	Wildcard in permission (`storage:*:read` , `settings:objects:*` , ...)	Wildcards are not supported — enumerate permissions in a comma list
400	"Invalid condition name for permission storage:logs:write : storage:bucket-name"	WHERE condition on a storage WRITE permission	Storage writes accept no conditions — grant unscoped or control via OpenPipeline
400	"Invalid permission name: spans:write / bizevents:write / buckets:write for service storage"	Permission doesn't exist	Only `logs` , `metrics` , `events` have storage `:write` ; spans/bizevents are ingest-only
400	"Invalid permission name: roles:admin for service environment"	No such role permission	Classic admin = `environment:roles:viewer` + `environment:roles:manage-settings`
400	"Invalid condition name ... storage:bucket.name"	Dotted condition field	The field is `storage:bucket-name` (hyphen)

HTTP Code	Error Message	Cause	Fix
400	"Policy binding cannot be created, such binding already exists" (also on the per-group update endpoint)	Binding already in desired state	Treat as success — the binding is converged (idempotent re-run)
400	"There is no policy binding to be deleted"	Deleting bindings from unbound policy	Safe to ignore during cleanup
400	"Duplicate entry"	Group with same name already exists	Reuse the existing group — look it up by name in `GET .../groups` (`.items[]`), as Script 1 does — or choose a different name
401	Unauthorized	Token is invalid or expired	Re-acquire OAuth token (they expire in ~5 min)
403	Forbidden	Token lacks required scopes	Add missing scopes to OAuth client
404	"Cannot get requested resource"	Wrong endpoint path or UUID	Check `/accounts/` vs `/repo/account/` for groups
500	"payload.map is not a function"	Same as 400 variant — array expected	Wrap group payload in array

Checklist: Nothing Happens After Group Creation

If your script creates groups but then silently stops:

1. **Check JSON escaping** — Policy `statementQuery` fields have nested quotes. Use `jq -n --arg` instead of string interpolation
2. **Check `set -e`** — With `set -e`, any non-zero exit stops the script silently. Use `set -uo pipefail` and handle errors explicitly, as Script 1 does: it counts failures, treats "already exists" as converged, and exits 1 at the end
3. **Check token expiry** — If group creation took time, the OAuth token may have expired before policy creation starts
4. **Test one policy manually** — Run the curl for a single policy creation and inspect the full response body

Checklist: Templated Policy Not Working

If users in a templated-policy group can't see data:

1. **Verify binding parameters** — Use the Report script (Section 3) to confirm `team=<value>` is set
2. **Verify security context** — Run the DQL query in Section 5 to confirm `dt.security_context` values match the binding parameter

3. **Check bucket assignment** — If using bucket-level policies, verify data is routed to the expected bucket via OpenPipeline
4. **Check evaluation order** — A DENY from another policy will override the ALLOW. Use the audit DQL to check for conflicting policies

Next Steps

- **IAM-10: Templated Policy Assignments** — Deep dive into template syntax and `${bindParam:}` patterns
- **IAM-11: Policy Persona Workshop** — Design the persona model that these scripts provision
- **IAM-04: Policy Authoring** — Full policy syntax reference
- **IAM-05: Boundary Design Patterns** — Data segmentation strategies

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.